

Algorithms for Data Science

Exercise 4

(with Solutions)

See the cheatsheet for valuable remarks!

Task 4.1: *Dynamic Programming: Nontouching Cells (1D)*

You are given an array consisting of n non-negative values. Your task is to choose a subset of the array cells such that no two cells touch and the total value is maximized. It suffices that you output the total value.

2	1	1	2	0
---	---	---	---	---

An example array ($n = 5$). The chosen cells are shaded in gray. The total value is 4.

Input: array $val[1..n]$.

Output: total value of the chosen array cells.

Solve the problem via the following methods. What is the running time and memory complexity?

(a) Backtracking

Solution: The idea is as follows: starting with $i = n$, we want to compute the best solution to the array $val[1..i]$. To do so, we check both options

- taking the i -th element, and
- not taking the i -th element

and we decide for the option that maximizes the objective.

But how to compute the objective for each of the two options?

- The best solution for the case that we don't take the i -th element is the same as the the best solution to the subarray $val[1..i - 1]$.
- The best solution for the case that we take the i -th element is the same as adding the value $val[i]$ of the i -th element to the best solution to the subarray $val[1..i - 2]$.

Altogether, we obtain the following algorithm.

```
AlgoBack( $val[1..n], i$ )
  if  $i == 0$  then
    return 0
  if  $i == 1$  then
    return  $val[1]$ 
  return max( $AlgoBack(val, i-1)$ ,  $val[i] + AlgoBack(val, i-2)$ )
```

The running time recurrence is $T(i) = T(i - 1) + T(i - 2) + \Theta(1)$. (Note that this recurrence cannot be solved with the master theorem.) The recurrence is the same as for the Fibonacci example in the lecture. Thus, the resulting running time is $\Theta(\varphi^n)$, which is exponential.

Given that the maximum depth of recursion is n , the recursion stack and, hence, the used memory, occupies $\Theta(n)$ space.

Further note: if we write “ $i \leq 0$ ” as the condition of the first if statement, then we do not need the second if statement at all. Thus, the code of our algorithm gets shorter and more elegant. I chose the variant above, as it will be later a little easier to transform it into a dynamic programming algorithm.

(b) Recursion with Memoization

Solution: We add a dictionary so that for each i , we compute the value only once.

```
memo = Init() // new dictionary
AlgoMem(val[1..n], i)
    if i == 0 then
        return 0
    if i == 1 then
        return val[1]
    if i not in memo then
        memo[i] = max(AlgoMem(val, i-1), val[i] + AlgoMem(val, i-2))
    return memo[i]
```

By implementing the dictionary via an array, we may assume that the access times to the dictionary are constant.

Thus, the total running time is in the order of the total number of recursive calls.

For each value of $i > 1$ there are two recursive calls but only the first time the function is called with this parameter. For subsequent calls or for values of $i \leq 1$, there are no recursive calls at all.

Thus, the total number of recursive calls (and hence the running time) is $\Theta(2(n - 1)) = \Theta(n)$, which is linear in n .

Concerning the memory complexity, the recursion stack has the same size as in the backtracking solution, that is, size $\Theta(n)$. Additionally, we have to store our results for n different values of i in our dictionary. Thus, also our dictionary needs $\Theta(n)$ space. Hence, the total space usage is $\Theta(n)$.

(c) Dynamic Programming

Solution: Basically, we do the solution of memoization but bottom up and without recursion.

```
AlgoDP(val[1..n])
    memo[0..n] = new array
    memo[0] = 0
    memo[1] = val[1]
    for i = 2 to n do
        memo[i] = max(memo[i - 1], val[i] + memo[i - 2])
    return memo[n]
```

In contrast to the two recursive solutions, it is fairly easy to determine the running time of our solution based on dynamic programming. The time is dominated by the time to create the auxiliary array as well as to complete the main for loop. Both take $\Theta(n)$ time in our case, hence, our third solution takes linear time in total.

The space complexity is now determined only by the size of the array $memo[0..n]$, hence, we use $\Theta(n)$ space. If we could make the array somehow smaller, we could save memory! Can we? Yes, see the next subtask!

(d) Dynamic Programming (with only $O(1)$ extra space!)

Solution: At any moment, we only need to know the two last cells of our *memo* array. By using the modulo trick, we can thus reduce the memory to a constant! Changes with respect to the last subtask are in blue.

```

AlgoDP(val[1..n])
    memo[0..1] = new array
    memo[0] = 0
    memo[1] = val[1]
    for i = 2 to n do
        memo[i%2] = max(memo[(i - 1)%2], val[i] + memo[(i - 2)%2])
    return memo[n%2]

```

The running time does of course not change. Since modulo operations are in reality relatively expensive (but still cost only a constant number of basic arithmetic operations), it is more efficient in practice (but in theory we see no difference) to rewrite the algorithm using only two variables. (Below, the operation $x, y = e, f$ assigns at the same time e to x and f to y .)

```

AlgoDP(val[1..n])
    b = 0
    a = val[1]
    for i = 2 to n do
        a, b = max(a, val[i] + b) , a
    return a

```

- (e) Modify the DP from subtask (c) such that it outputs the chosen grid cells (**Output:** binary array $chosen[1..n]$ where $chosen[i] = true$ if the grid cell in the i -th column has been chosen.)

Solution: Run the DP from subtask (c) to compute the DP table, and then go backwards through the array checking each time what was the best option: to take or not to take the i -th element.

```
AlgoDP( $val[1..n]$ )
     $memo[0..n] = \text{new array}$ 
     $memo[0] = 0$ 
     $memo[1] = val[1]$ 
    for  $i = 2$  to  $n$  do
         $memo[i] = \max(memo[i - 1], val[i] + memo[i - 2])$ 

     $chosen[1..n] = \text{new array filled with 0s.}$ 
     $i = n$ 
    while  $i > 1$  do
        if  $memo[i] == val[i] + memo[i - 2]$  then
             $chosen[i] = 1$ 
             $i = i - 2$ 
        else
             $i = i - 1$ 
    if  $i == 1$  then
         $chosen[1] = 1$ 
    return  $chosen$ 
```

Task 4.2: Dynamic Programming: Nontouching Cells (2D)

Consider the following generalization of the last task (Task 4.1). You are given a $(k \times n)$ -grid consisting of n columns and k rows. Each grid cell has some non-negative value. Your task is to choose **at most one** grid cell from each of the n columns such that no two grid cells are touching each other (touching with corners is also forbidden) and the total value is maximized. It suffices that you output the total value.

0	1	2	1	0
1	1	1	1	0
0	1	2	1	2
0	1	2	1	2

An example grid ($n = 5$ and $k = 4$). The chosen cells are shaded in gray. The total value is 6.

Input: array $val[1..n][1..k]$.

Output: total value of the chosen grid cells.

Solve the problem for any $k \geq 4$ via the following methods. What is the running time?

(a) Backtracking

Hint: First solve the problem for the restricted case where the input specifies some $j \in \{1, \dots, k\}$ and, within the *last* column, we are allowed to pick only the grid cell of the j -th row.

Solution:

Running time: Exponential in n (in more detail: $k^{\Theta(n)}$) as for each column we have $2k$ recursive calls.

```

/* We call our function below trying all combinations for the last
   column. */
maxVal = 0
for j = 1 to k do
    maxVal = max(maxVal, AlgoBack(val, n, j))
return maxVal

```

```

/* The output is the maximum value attainable in val[1..i][1..k] under the
   condition that from the i-th column we may take only the
   element val[i][k]. */
AlgoBack(val[1..n][1..k], i, j)
    if i == 0 then
        return 0
    maxVal = 0
    // The case that we don't take val[i][j].
    for t = 1 to k do
        maxVal = max(maxVal, AlgoBack(val, i-1, t))
    // The case that we take val[i][j].
    for t = 1 to k do
        // The cell val[i-1][t] touches val[i][j] if t ∈ {j-1, j, j+1}.
        // As k > 3, there is at least one non-touching cell.
        if t < j-1 or t > j+1 then
            maxVal = max(maxVal, val[i][j] + AlgoBack(val, i-1, t))
    return maxVal

```

(b) Recursion with Memoization

Solution: We just add a dictionary *memo* to the last solution. The modifications are in blue.

```
memo = Init()
AlgoMem(val[1..n][1..k], i, j)
    if i == 0 then
        return 0
    if (i,j) not in memo then
        maxVal = 0
        for t = 1 to k do
            maxVal = max(maxVal, AlgoMem(val,i-1,t))
        for t = 1 to k do
            if t < j - 1 or t > j + 1 then
                maxVal = max(maxVal, val[i][j] + AlgoMem(val,i-1,t))
        memo[i][j] = maxVal
    return memo[i][j]
```

For this problem, we can assume that the dictionary is a two-dimensional array *memo*[1..n][1..k] (hence, we use the dictionary type “Array indexed by keys”: initially, we fill the array with a value that never appears as a solution, for instance with -1 . Thus, if we get the query “Is (i, j) **not in** *memo*?”, we answer with true if and only if *memo*[*i*][*j*] == -1 . Thus, the access and modification times for *memo* are constant in such a solution. The time to initialize such a dictionary is $\Theta(nk)$.

To analyze the total running time, there are two possibilities:

First possibility. Observe that the running time is asymptotically equal to the total number of recursive calls. To see it, you can argue as follows: charge the time of the for loops (where we make the recursive calls) to the functions that are recursively called. Consequently, the total time charged to each function call can be simplified to just $\Theta(1)$ (the time for the code besides the for loops—note that the time to access our dictionary is also $\Theta(1)$). Consequently, the total running time is just the number of recursive calls times $\Theta(1)$. Note that, within *AlgoMem*(*val*,*i*,*j*), $\Theta(k)$ recursive calls are made but only if $i \geq 1$ and only for the first time *AlgoMem*(*val*,*i*,*j*) is run for the parameters *i* and *j*. For any subsequent runs (or if $i = 0$), there are no recursive calls at all within *AlgoMem*(*val*,*i*,*j*). Therefore the total number of recursive calls is asymptotically equal to $\Theta(k)$ times the number of all combinations of $i \geq 1$ and *k*. Thus, the resulting running time (together with the time to initialize the dictionary) is $\Theta(nk^2)$.

Second possibility. The first time we call our function for a given combination of *i* and *j*, the running time is $\Theta(k)$ (the for loops) plus the time for the subsequent recursive calls (with the exception of $i = 0$, then the time is just $\Theta(1)$). For any subsequent calls of our function with the same combination of *i* and *j*, the running time is just equal to the time to access our dictionary, which is $\Theta(1)$ given the choice of our dictionary.

So, the total running time consists of three parts:

1. the total running time of first-time calls of our function,
2. the total running time of subsequent calls of our function, and
3. the time $\Theta(nk)$ to initialize our dictionary in the beginning.

The first part is just the number of all combinations of $i > 0$ and $j \geq 0$ times $\Theta(k)$ (plus $\Theta(1)$ times the number of combinations of $i = 0$ and $j \geq 0$). Hence, we get $\Theta(nk^2)$ for the first part in total.

The second part is not larger than the number of all recursive calls time $\Theta(1)$. In total, for each combination of i and j , we make at most k recursive calls. Thus, there are at most $O(nk^2)$ recursive calls. Hence, the running time of the second part is $O(nk^2)$ (in Oh-notation as we are lazy and argued only about the upper bound, which is enough as the first part takes already $\Theta(nk^2)$).

Altogether, we obtain $\Theta(nk^2)$ as the resulting running time.

(c) Dynamic Programming

Solution: We do everything bottom up. Time $\Theta(nk^2)$, Space $\Theta(nk)$.

```

DPtable[0..n][1..k] = new array filled with 0s
for i = 1 to n do
    for j = 1 to k do
        maxVal = 0
        for t = 1 to k do
            maxVal = max(maxVal, DPtable[i - 1][t])
        for t = 1 to k do
            if t < j - 1 or t > j + 1 then
                maxVal = max(maxVal, val[i][j] + DPtable[i - 1][t])
            DPtable[i][j] = maxVal
    maxVal = 0
    for t = 1 to k do
        maxVal = max(maxVal, DPtable[n][t])
return maxVal

```

- (d) Dynamic Programming with $O(k)$ extra memory

Solution: Just record the current and the last column (in total two at any moment). Changes are in blue. If i is odd, then we use the column 1 in the DP table to store values for it, otherwise the column 0; hence, $i\%2$ gives us the column for i (where $i\%2$ means i modulo 2).

```

DPtable[0..1][1..k] = new array filled with 0s
for i = 1 to n do
    for j = 1 to k do
        maxVal = 0
        for t = 1 to k do
            maxVal = max(maxVal, DPtable[(i - 1)%2][t])
        for t = 1 to k do
            if t < j - 1 or t > j + 1 then
                maxVal = max(maxVal, val[i][j] + DPtable[(i - 1)%2][t])
            DPtable[i%2][j] = maxVal
    maxVal = 0
    for t = 1 to k do
        maxVal = max(maxVal, DPtable[n%2][t])
return maxVal

```

- (e) (Bonus Task) Modify the algorithm of task (c) such that it outputs the chosen grid cells as a binary array $chosen[1..n][1..k]$ where $chosen[i][k] = true$ if the grid cell in the i -th column and j -th row has been chosen.

Solution: When finished computing the DP table go back through the table and write into $chosen$ which element you choose.

Input: $maxVal$ and $DPtable[0..n][1..k]$ as computed by our dynamic program.

Output: $chosen[1..n][1..k]$

```

// To make our algorithm simpler, we assume that we have a dummy element
// in the non-existing column  $n+1$  at the non-existing row  $k+2$  (that
// does not touch any other cells of the actual input table). Starting
// from this dummy element, the current candidate has coordinates  $(i, j)$ 
// and we want to find out whether to add it or not.
 $chosen[1..n+1][1..k]$  = new boolean array; initially all entries set to false; // for
// technical reasons  $n+1$  columns
 $i = n+1$ 
 $j = k+2$ 
 $val[i][j] = 0$  // Later, when  $i \leq n$ , it will be equal to  $val[i][j]$ .
while  $i \geq 1$  do
     $notFound = true$  // Is set to true as long as we don't know from which
    // cell and in what case we reached  $(i, j)$ 

    // Case 1: We don't take element  $(i, j)$ 
     $t = 1$ 
    while  $t \leq k$  and  $notFound$  do
        if  $maxVal == DPtable[i-1][t]$  then
             $notFound = false$ 
             $j = t$ 
         $t = t + 1$ 

    // Case 2: We take element  $(i, j)$ . This case must hold if  $notFound$  is
    // still set to false.
     $t = 1$ 
    while  $notFound$  do
        if  $(t < j - 1$  or  $t > j + 1)$  and  $maxVal == val[i] + DPtable[i-1][t]$  then
             $notFound = false$ 
             $chosen[i][j] = true$  // Note that the  $j = k+2$  value of the first
            // candidate would lead to an error as  $chosen$  has only
            // size  $[1..n+1][1..k]$ . However, Case 1 is always satisfied for
            // the first candidate; thus, we will never reach this line of
            // code with the first candidate.
             $j = t$ 
         $t = t + 1$ 

    // The next candidate is at column  $i = i - 1$  and at row  $j$  (we computed  $j$ 
    // in Case 1 or 2)
     $i = i - 1$ 
     $val[i] = val[i][j]$ 
     $maxVal = DPtable[i][j]$ 

Remove the last (technical) column from  $chosen$ 
// Now,  $chosen$  has the same size as  $val$ , that is, size  $[1..n][1..k]$ .
return  $chosen$ 

```

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 4.3: Dynamic Programming: Maximum Block

Given an array containing possibly negative values, compute the value of a block of maximum total value.

Formally, let $a[1..n]$ be the input array. The problem is to output the total value of a *heaviest* block, that is, of a subarray $a[i..j]$ of maximum total value $\sum_{k=i}^j a[k]$.

Solution: *The solutions are actually very simple (see any of the final algorithms below). In the following text, I give you some intuition on how to come up with the final solution in two different ways.*

Solution 1 (Start with Recursion): Before you try recursion, you might try the very direct approach of just computing

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a[k] .$$

```
AlgoSlow(a[1..n])
  max_sum = 0
  for i = 1 to n do
    for j = i to n do
      sum = 0
      for k = i to j do
        sum = sum + a[k]
      if sum > max_sum then
        max_sum = sum
  return max_sum
```

The time is $\Theta(n^3)$ and space complexity is $\Theta(1)$. Space complexity is optimal and the time is polynomial. Thus, the time is not so bad, but is it possible to do it better? If we think for short while, we notice, that in the inner most loop we recompute some parts over and over again. After some thinking, we might end up with the following $\Theta(n^2)$ -time algorithm that again has optimal $\Theta(1)$ space complexity.

```
AlgoFaster(a[1..n])
  max_sum = 0
  for i = 1 to n do
    sum = 0
    for j = i to n do
      sum = sum + a[j]
      if sum > max_sum then
        max_sum = sum
  return max_sum
```

But is this quadratic running time optimal?

Let's try a recursive approach using divide and conquer! In the direct approach above, *max_sum* was the best solution found so far in the subarray $a[1..i]$. So, let's also use such a parameter i . If i is 0, we define the array to be empty and return 0. Otherwise, we reduce the problem to a smaller one. We have one decision to make: to take or not to take $a[i]$ into our block. If we take it, then we either have also to take the next element, $a[i-1]$, or to stop the recursion (meaning, that $a[i]$ begins the block) as the block has to consist of consecutive elements. Thus, we will have also a second boolean parameter *take_or_stop*.

- If it is **False**, it means we haven't started a block yet, and we can decide whether we skip the i -th element, or whether we take it and start a block. In our algorithm, we check both cases recursively and return the maximum.
- If *take_or_stop* is **True**, we have to either stop the recursion or take the i -th element. Again, we check both cases and take the maximum of 0 (stopping the recursion) and the recursively obtained value when choosing $a[i]$.

We call the function with the parameters $i = n$ and *take_or_stop* = **False**.

```

AlgoRec( $a[1..n]$ ,  $i$ , take_or_stop)
┌   if  $i == 0$  then
├       return 0;
└   else
    ┌   if take_or_stop then // if take_or_stop is True
    └       return max(0,  $a[i] + \text{AlgoRec}(a, i-1, \text{True})$ )
    ┌   else
    └       return max( $\text{AlgoRec}(a, i-1, \text{False})$ ,  $a[i] + \text{AlgoRec}(a, i-1, \text{True})$ )

```

Surprisingly, the running time is not exponential as in many other recursive approaches, but $\Theta(n^2)$ —thus as good as our best direct approach above! To see the quadratic running time, take a sheet of paper and draw a node for each combination of the two parameters. Draw an arrow (starting nowhere) to the node with $i = n$ and *take_or_stop* = **False**. Next, if some node A has t incoming arrows, draw t arrows from A to each node that is called by A . The total number of arrows is now asymptotic to the total running time. You will notice that nodes with the parameter *take_or_stop* = **False** have exactly one incoming arrow whereas nodes with *take_or_stop* = **True** have exactly $n - i$ incoming arrows. Thus, the number of arrows is

$$\Theta\left(n + \sum_{i=0}^n (n - i)\right) = \Theta\left(n + \sum_{i=0}^n i\right) = \Theta(n + n^2) = \Theta(n^2) .$$

Since the running time of the recursive approach is already so good, will it become even better if we add memoization?

```

mem = Init() // new dictionary
AlgoMem(a[1..n], i, take_or_stop)
    key = (i, take_or_stop)
    if key not in mem then
        if i == 0 then
            mem[key] = 0
        else
            if take_or_stop then // if take_or_stop is True
                mem[key] = max(0, a[i] + AlgoMem(a, i-1, True))
            else
                mem[key] = max(AlgoMem(a, i-1, False), a[i] + AlgoMem(a, i-1, True))
    return mem[key]

```

Now, what is the running time? If we take a picture with the nodes that we did above, we notice that suddenly each node sends exactly one arrow (for $i > 0$). Thus, the total number of arrows is the number of nodes which is the number of combinations of i and true/false values, which is $\Theta(n)$. And this is our running time! Nice, we achieved linear running time using recursion with memoization. So is this algorithm better than the direct approach above? In terms of time: yes, definitely! In terms of memory: no! Our algorithm with memoization uses $\Theta(n)$ extra space which is much worse when compared to the constant space used by our direct approach above.

Is there any hope to use less space? Yes, by formulating a dynamic programming algorithm and then trying to reduce the space.

By going the recursive approach backwards from bottom ($i = 0$) to top ($i = n$), we get the following dynamic programming solution using a two-dimensional array $dp[0..n][0..1]$. Our goal is to directly compute dp such that that $d[i][b]$ equals $mem[(i, b)]$ (where we interpret $b = 0$ as **False** and $b = 1$ as **True**).

```

AlgoDP(a[1..n])
    dp[0..n][0..1] new array filled with 0s. for i = 1 to n do
        dp[i][1] = max(0, a[i] + dp[i-1][1])
        dp[i][0] = max(dp[i-1][0], a[i] + dp[i-1][1])
    return dp[n][0]

```

It's easy to see that time and space is $\Theta(n)$. Can we reduce the space now? Observe that $dp[i][0]$ and $dp[i][1]$ depend only on position $i - 1$ of the DP array. Hence, using the modulo trick, we can shrink the DP table to $dp[0..1][0..1]$ and thus obtain $\Theta(1)$ space.

```

AlgoDP(a[1..n])
    dp[0..1][0..1] new array filled with 0s. for i = 1 to n do
        dp[i%2][1] = max(0, a[i] + dp[(i-1)%2][1])
        dp[i%2][0] = max(dp[(i-1)%2][0], a[i] + dp[(i-1)%2][1])
    return dp[n%2][0]

```

Note that we actually do not need to keep $dp[i\%2][0]$ and $dp[(i-1)\%2][0]$ at the same time in the memory. By rewriting the code a little bit by using an auxiliary variable *temp*, we neither need to

keep $dp[(i-1)\%2]$. Thus, we can shrink the table more do $dp[0..0][0..1]$, meaning that it becomes a one-dimensional table $dp[0..1]$ independent of i .

```
AlgoDP( $a[1..n]$ )
┌    $dp[0..1]$  new array filled with 0s. for  $i = 1$  to  $n$  do
│   ┌    $temp = a[i] + dp[1]$ 
│   │    $dp[1] = \max(0, temp)$ 
│   └    $dp[0] = \max(dp[0], temp)$ 
└   return  $dp[0]$ 
```

Instead of using the table, it's more efficient to replace it by two variables. We can even get rid of the $temp$ variable, optimizing the code down to two variables in total (not counting the iterator i).

```
AlgoDP( $a[1..n]$ )
┌    $c = 0, d = 0$ 
│   for  $i = 1$  to  $n$  do
│   ┌    $d = a[i] + d$ 
│   │    $c = \max(c, d)$ 
│   └    $d = \max(0, d)$ 
└   return  $c$ 
```

Note that we can further shorten the code by taking the maximum with 0 already in the first line of the loop.

```
AlgoDP( $a[1..n]$ )
┌    $c = 0, d = 0$ 
│   for  $i = 1$  to  $n$  do
│   ┌    $d = \max(0, a[i] + d)$ 
│   │    $c = \max(c, d)$ 
└   return  $c$ 
```

Thus, starting with the recursive approach, we ended up with a very efficient and elegant iterative approach (with time $\Theta(n)$ and space $\Theta(1)$).

Solution 2 (Start directly with DP): How to solve such a problem using recursion or dynamic programming? Let us take a look at what choices we have:

- If we choose to take some element $a[j]$ into the block, then (i) either $a[j]$ is the first element of the block—meaning that we cannot take any element $a[k]$ with $k < j$ to the block—, or (ii) $a[j]$ is some middle element meaning that we have to choose $a[j - 1]$ also to the block and to again consider the two options (i) and (ii) for the element $a[j - 1]$ (hence, we have a recursion here!).

By this reasoning we can compute the maximum total value of a block within the subarray $a[1..j]$ assuming that the block ends in $a[j]$; let us call this value $DPtake[j]$. In case (i), the block also starts in $a[j]$, hence, consists only of $a[j]$ and has therefore value just $a[j]$. In case (ii), the block consists of $a[j]$ and—this is now the important observation—of a heaviest block that ends in $a[j - 1]$, that is, of value $DPtake[j - 1]$. Hence, to compute $DPtake[j]$, we have to take the maximum over both cases (i) and (ii):

$$DPtake[j] = \max(a[j], a[j] + DPtake[j - 1]) .$$

Now, the question arises: Couldn't we just write $DPtake[j] = a[j] + DPtake[j - 1]$? The answer is no as $DPtake[j - 1]$ might be negative—recall that we have negative numbers!

- OK, but what if we don't choose to take the element $a[j]$ into the block? Then we have to find a block to the left and a block to the right of j of total maximum value. Let $DP[k]$ be the value of a heaviest block in the subarray $a[1..k]$ independently of whether $a[k]$ has been taken). Then the heaviest block to the left of j has value $DP[j - 1]$. Let us skip the right side for now and ask, how to compute $DP[j]$.

Above, we have seen two choices, to take or not to take $a[j]$. What is the better choice? If we are interested in a heaviest block in the subarray $a[1..j]$ (whose value is defined to be $DP[j]$), then the answer is to take the maximum over both choices:

$$DP[j] = \max(DPtake[j], DP[j - 1]) .$$

But what about the right side, that is, the subarray $a[j + 1..n]$? Well, at the end of the day, we are interested in computing $DP[n]$ which is in fact the answer to the overall problem: the total value of a heaviest block in $a[1..n]$! Therefore, if we are able to compute $DP[n]$ without considering the right sides for particular values for j , we are totally fine!

Since $DP[j]$ depends only on $DP[j - 1]$ and $DPtake[j]$, whereas $DPtake[j]$ depends only on $DPtake[j - 1]$ and $a[j]$, we can compute $DP[n]$ by computing the respective values from left to right (for increasing j) or in a recursive manner.

Our discussion leads directly to the following dynamic programming algorithm:

```

Block_DP( $a[1..n]$ )
     $DP[0..n], DPtake[0..n]$  = new arrays filled with zeros
    for  $j = 1$  to  $n$  do
         $DPtake[j] = \max(a[j], a[j] + DPtake[j - 1])$ 
         $DP[j] = \max(DPtake[j], DP[j - 1])$ 
    return  $DP[n]$ 

```

Since the for loop repeats only n times and has only operations that take constant time, the total running time is $\Theta(n)$. The space complexity is $\Theta(n)$ due to the two arrays DP and $DPtake$. However, we can even reduce the space complexity to $\Theta(1)$ by observing that for computing the values for index position j , we only need the index value of position $j - 1$. Thus, arrays of size two suffice:

```

Block_DP_lessspace( $a[1..n]$ )
     $DP[0..1], DPtake[0..1]$  = new arrays filled with zeros
    for  $j = 1$  to  $n$  do
         $DPtake[j\%2] = \max(a[j], a[j] + DPtake[(j - 1)\%2])$ 
         $DP[j\%2] = \max(DPtake[j\%2], DP[(j - 1)\%2])$ 
    return  $DP[n\%2]$ 

```

If we look more closely, then we even notice that we can discard the values for position $j - 1$ in the moment we finish computing the values for position j . Hence, we do not need an array at all, but two variables c and d suffice:

```

Block_DP_lessspace( $a[1..n]$ )
     $c = 0, d = 0$ 
    for  $j = 1$  to  $n$  do
         $d = \max(a[j], a[j] + d)$ 
         $c = \max(d, c)$ 
    return  $c$ 

```

To make the picture complete, we also discuss the recursive solutions. We begin by solving the problem recursively via backtracking. We have one method corresponding to DP and another corresponding to $DPtake$. We call the algorithm via $\text{Block_Back}(a, n)$.

```

Block_Back( $a[1..n], j$ )
    if  $j == 0$  then
        return 0
    return  $\max(\text{Block\_Back}(a, j-1), \text{Blocktake\_Back}(a, j))$ 

Blocktake_Back( $a[1..n], j$ )
    if  $j == 0$  then
        return 0
    return  $\max(a[j], a[j] + \text{Blocktake\_Back}(a, j-1))$ 

```

The algorithm above has a high running time but—interestingly—it is not too horrible. Observe that $\text{Block_Back}(a, j)$ is called exactly once for every value of j (and has constant time per call—not counting the time within the recursive calls). Thus, the total running time alone for all calls

of `Block_Back()` is $\Theta(n)$. What about `Blocktake_Back(a,j)`? For each j , it is called $n-j+1$ times: once by `Block_Back(a,j)` and $n-j$ times by `Blocktake_Back(a,j+1)`. The total running time is therefore $\Theta(\sum_{j=1}^n (n-j+1))$ which can be shown to be $\Theta(n^2)$. Hence, not so bad.

But, we can do it better using memoization and improve the running time down to $\Theta(n)$. We use two dictionaries, called *DP* and *DPtake*.

```

DP = Init(), DPtake = Init()
Block_Mem(a[1..n],j)
    if j == 0 then
        return 0
    if j not in DP then
        DP[j] = max(Block_Mem(a,j-1), Blocktake_Mem(a,j))
    return DP[j]
Blocktake_Mem(a[1..n],j)
    if j == 0 then
        return 0
    if j not in DPtake then
        DPtake[j] = max(a[j], a[j] + Blocktake_Mem(a,j-1))
    return DPtake[j]

```

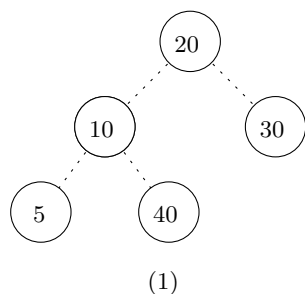
Task 4.4: Heap Sort

Run heap sort on the array $a = \langle 20, 10, 30, 5, 40 \rangle$. Draw the array a after each single change as a binary complete tree.

Solution: First, the array is transformed into a heap. Here, we do it by adding the elements one after the other. One could also call the method `BuildHeap()` from the literature which would result in a different heap tree. After obtained the heap, we remove the maximum element until the heap is empty.

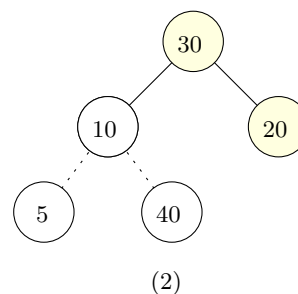
For clarity, changes are marked in yellow and edges that do not belong to the heap are dashed.

Beginning:



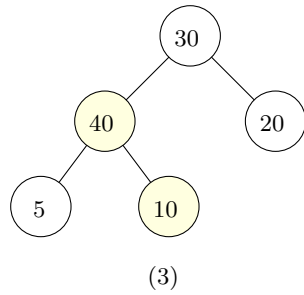
No change after `Add(20)` and `Add(10)`

Change after `Add(30)`

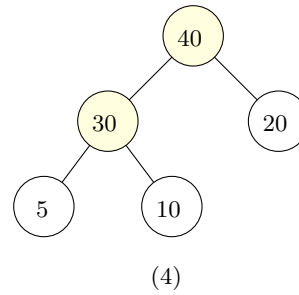


No change after **Add(5)**

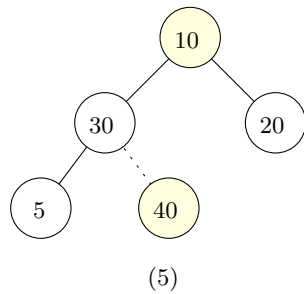
First change during **Add(40)**



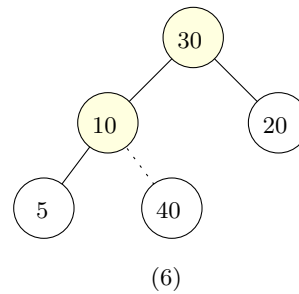
Secong (last) change during **Add(40)**



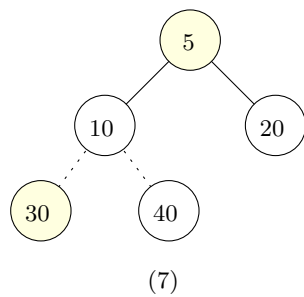
First change during first **RemoveMax()**



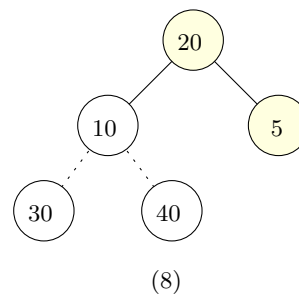
Second (last) change during first **RemoveMax()**

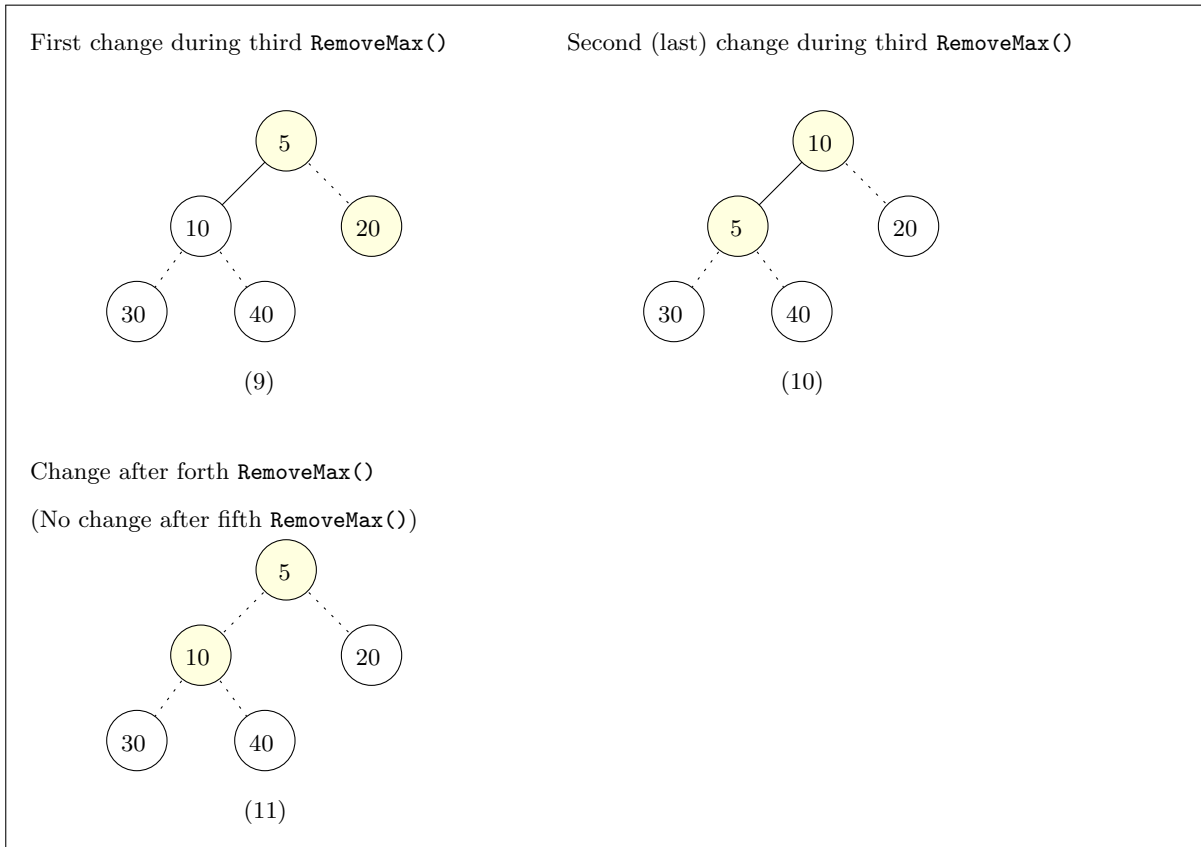


First change during second **RemoveMax()**



Second (last) change during second **RemoveMax()**





Task 4.5: Quick Sort: Polish Flag

Second May is Polish National Flag Day. For this occasion, you want to program a robot to arrange red and white balls to form a Polish flag. In particular, you are given $2n$ bins numbered from 1 to $2n$ where each bin contains exactly one ball. In total there are n red and n white balls. Your goal is that the first n bins contain red balls, the remaining ones white balls. Your robot can only do the following operations:

- `isRed( $i$ )` : the robot looks at the i -th bin and returns true if it contains a red ball, and false otherwise.
- `swap( $i, j$ )`: the robot exchanges the balls of the i -th and j -th bin.

Furthermore, your robot has only a small memory of constant size.

(a) How is this problem related to quick sort?

Solution: The Polish flag problem is equivalent to the subproblem of quick sort where elements smaller than the pivot (let's call them red) have to be moved to the left side of the pivot and elements larger than the pivot (let's call them white) have to be moved to the right side of the pivot. For instance, if we color our elements like this, then any algorithm for the Polish flag problem will move the elements correctly. At the end, we just have to reinsert the pivot element in between them. On the other hand, the quick sort subroutine would solve the Polish flag problem directly if we define the order *red is smaller than white* (or, alternatively, give all red balls the value 0 and all white balls the value 1).

Hence, if we can solve the Polish flag problem using only *a small memory of constant size* as required, then the quick sort algorithm for the subproblem, and in fact, for the overall sorting

problem, can also be implemented by using only constant additional memory. In the next subtask, we observe that this is in fact possible.

- (b) Program the robot to arrange a Polish flag within $O(n)$ operations.

Solution:

```
left = 1
right = n
// The invariant: Anything to the left of left is red, and anything to
// the right of right is right.
while left < right do
    // The exclamation mark in !isRed(left) negates the output. That is,
    // the following if statement should be read as:
    // If isRed(left) == false and isRed(right) == true
    if !isRed(left) and isRed(right) then
        swap(left, right)
    if isRed(left) then
        left = left + 1
    if !isRed(right) then
        right = right - 1
```

The algorithm terminates as, in each repetition of the while loop, we increase *left* or decrease *right*. Thus, we repeat the for loop at most $n - 1$ times and therefore the running time is linear in n .

The invariant is correct as we increase or decrease *left* and *right* only if they are red or white, respectively. Consequently, the output is correct.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 4.6: Fibonacci Numbers

We define the Fibonacci numbers as follows:

$$F_n = \begin{cases} n & n = 0, 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases}$$

- (a) How many times do we perform an addition if compute F_n recursively with the formula above?

Solution: Asymptotically, there are $\Theta(F_n)$ additions because if we resolve the recurrence, we get a sequence of zeros and ones with plus symbols in between. Obviously, the number of ones is exactly F_n , whereas the number of zeros is at most F_n as each zero appears in a sum with a one. Thus, there are at least $F_n - 1$ and at most $2F_n - 1$ additions.

The exact number of additions is $F_{n+1} - 1$. One can prove this by mathematical induction.

- (b) Give an algorithm that computes F_n using only $O(\log n)$ arithmetic operations. Use the following recurrence:

$$\text{For } n > 1, \quad F_{2n-1} = F_n^2 + F_{n-1}^2 \quad \text{and} \quad F_{2n} = F_n^2 + 2F_n F_{n-1} .$$

Solution: Observation: The given recurrence implies that all the three numbers F_{n-1} , F_n and F_{n+1} can be computed by using only the three numbers $F_{\lfloor n/2 \rfloor - 1}$, $F_{\lfloor n/2 \rfloor}$ and $F_{\lfloor n/2 \rfloor + 1}$. Thus, one can design a recursive algorithm that on input n computes these three values by doing a single recursive call on $\lfloor n/2 \rfloor$. After a logarithmic number of calls, we reach the base case $n = 0, 1$.

- (c) Give an algorithm that computes F_n using only $O(\log n)$ arithmetic operations. Use the following matrix recurrence:

$$\text{For } n > 1, \quad \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix} .$$

Solution: The recurrence implies the following equation:

$$\text{For } n > 1, \quad \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix} .$$

Thus, the problem boils down to compute the matrix $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1}$ using $O(\log n)$ arithmetic operations (addition and multiplication). This can be done recursively using the following trivial recurrence for a matrix M :

$$M^{2n} = M^n M^n \quad \text{and} \quad M^{2n+1} = M M^n M^n .$$

Note: In the RAM model, each basic arithmetic operation costs $O(1)$ time. This is realistic as long as the values are not too large. However, in the Fibonacci algorithms above such an assumption is not realistic as the n -th Fibonacci number has $\Theta(n)$ bits in its standard encoding, and consequently it takes $\Theta(n)$ time just to write it down, not to mention the time to compute it. Thus, to be more realistic, we should assume that the addition of two n -bit numbers takes $\Theta(n)$ time, whereas multiplication takes $O(n \log n)$ time. (It is an open question in computer science whether one can do the multiplication faster. The best-known $O(n \log n)$ algorithm for multiplication is relatively young from 2019.) Summarized, the best-known algorithm to compute the n -Fibonacci number takes $O(n \log^2 n)$ time.